CAMP: Consumption-Aware Memory Prediction for Scientific Workflows

Nagarjuna Chowdary Kandimalla, Bhanu Prakash Vangala, and Tanu Malik
Department of Electrical Engineering and Computer Science
University of Missouri
Columbia, MO 65211, USA
{nkmh5@umsystem.edu, bv3hz@missouri.edu, tanu@missouri.edu}

***Abstract*—Distributed scientific workflow tasks exhibit significant variation in memory consumption, making accurate peak memory estimation critical for resource management and avoiding out-of-memory (OOM) failures. Existing workload schedulers rely on static features like file sizes or tool names, which fail to capture non-linear memory scaling. We present Consumption-Aware Memory Prediction (`CAMP`), a closed-loop framework that utilizes fine-grained, consumption-level provenance—the exact number of bytes a task reads—measured dynamically via eBPF kernel probes. To minimize tracing overhead across parallel task executions at scale, we develop an active-learning-based, multi-gate selective auditing policy that samples only the most informative workflow tasks for model retraining. We evaluate `CAMP` on thousands of production-scale bioinformatics workflow records executed under Slurm. Compared to size-based scheduling baseline, incorporating dynamic data-consumption features reduces prediction median absolute percentage error by up to 40.31%. Furthermore, `CAMP` successfully reduces predicted OOM events by 64.37% while slashing runtime-weighted memory wastage by 37.18% on our consumption unaware baseline. Our selective auditing framework consistently outperforms random sampling in identifying high- risk and informative tasks across tight profiling budgets, proving that kernel-level consumption data provides a highly scalable and practical feedback signal for high-performance workflow resource allocation.**

***Index Terms*—scientific workflows, memory prediction, eBPF, selective auditing, Slurm**

I. Introduction

Scientific workflows in genomics, remote sensing, and materials science submit thousands of tasks to Slurm [1], HTCondor [2], and Kubernetes [3], which require memory reservations before execution. Underestimation causes out-of-memory (OOM) failures, restarts, and disrupted dependencies [4], [5]; overestimation reduces utilization and increases queuing delays [4], [6]. Uncertain footprints of black-box tools across datasets encourage conservative reservations [5], [6].

Prior work emphasizes model selection: k-nearest neighbors, random forests, and gradient-boosted trees capture complex runtime behavior better than linear regressors [6], [7]. Yet prediction also depends on prelaunch information, with two key limitations.

First, static features—staged file sizes, process names, and thread counts [4]–[6], [8]—do not reveal actual consumption. Selective queries may read little of a large index; memory-mapped tools load segments through page faults without conventional `read` calls. Staged volume thus differs from data contributing to resident memory. Fine-grained consumption measurements capture this distinction, but continuous tracing is costly and current-task measurements are unavailable at scheduling time.

Second, streaming workloads have uneven history: frequent workflow steps have many similar past runs, while novel or updated stages lack empirical baselines. A framework must preserve compatible history at multiple levels of specificity, falling back to broader evidence when local support is sparse.

This paper presents `CAMP`, a closed-loop memory-allocation framework designed to address these feature limitations. `CAMP` utilizes an eBPF-based auditing daemon to dynamically capture explicit read bytes and file-backed memory-mapped (`mmap`) page faults, directly attributing these I/O operations to individual task process trees. Completed tasks populate an asynchronous database, supplying peak RSS records globally and kernel-level data consumption metrics for instrumented runs. To exploit historical data at the appropriate granularity, `CAMP` structures empirical observations into hierarchical compatibility *scopes*, ranging from specific instance repetitions to global hardware fallbacks. At prelaunch, `CAMP` uses these histories to predict the pending task's consumed data, estimate central and upper-tail peak memory, and calibrate a memory request.

To bound tracing overhead on production clusters, `CAMP` operates under a strict administrative profiling budget, deploying a multi-gate active learning selection policy. These gates prioritize tasks with a high probability of allocation failure, high anticipated training information utility, or those operating within under-observed feature regions. The resulting system evaluates a continuous loop of multi-quantile prediction, selective auditing, kernel measurement, and asynchronous retraining. The primary contributions of this work are:

- a predictive allocation architecture combining *scope*-based hierarchical history summaries, consumption-aware quantile prediction, and residual calibration;
- an evaluation of fixed multi-gate selective auditing policies against uniform and process-stratified random baselines under equivalent sampling budgets;
- a comprehensive experimental evaluation on several Bioinformatics workflows including comparative performance baseline against Sizey [6]; and a live Bowtie2 case study in the technical report [9].

TABLE I
NOTATION USED IN THE PROBLEM FORMULATION.

Symbol	Description
$t = 1, \ldots, N$	Sequential task index
$\mathcal{P}(t)$	Tasks completed before task t launches
$\mathbf{s}_t$	Static features, known at launch
a_t	Aggregate staged input size (part of $\mathbf{s}_t$)
c_t	Bytes consumed at runtime; observable only if audited
y_t	True peak memory (max RSS), known after completion
$\hat{y}_t$	Memory request issued at launch
$\mathbf{h}_{m,t}$	Summary of peak memory over $\mathcal{P}(t)$
$\mathbf{h}_{c,t}$	Summary of consumption over audited tasks in $\mathcal{P}(t)$
$\mathbf{h}_t$	Combined history $[\mathbf{h}_{m,t}, \mathbf{h}_{c,t}]$
$q_t \in \{0, 1\}$	Auditing decision for task t
$B \in (0, 1]$	Auditing budget (maximum audited fraction)
f, π	Allocation function; auditing policy

To the best of our knowledge this is the first time a distribution based model is used for peak memory prediction. Past work [6] has predominantly considered point-based models.

Sections II and III present the problem and CAMP design. Sections IV and V describe the experimental methods and results, followed by related work (Section VI) and conclusions (Section VII).

II. Problem Description

A workflow declares a set of processes (pipeline steps); each process is instantiated as many tasks, one per input shard. Tasks of the same workflow–process pair execute identical code on different inputs.

In CAMP, we assume a workflow executes tasks $t = 1, \ldots, N$ arriving sequentially. Before launch, a predictive framework must issue a memory request $\hat{y}_t$ for task t. Each task is associated with:

- **Static features** ($\mathbf{s}_t$): a feature vector available at prelaunch scheduling time, including workflow metadata, process type, thread count, node constraints, and the aggregate staged input size $a_t = \sum_{f \in \text{staged}(t)} \text{size}(f)$.
- **Dynamic consumption** (c_t): the number of bytes the task consumes during execution. Although highly predictive of peak memory, it is observable only through kernel-level instrumentation and is therefore hidden at launch ($c_t \notin \mathbf{s}_t$).
- **Target peak memory** (y_t): the maximum resident set size (RSS), measured after execution completes.

Because tasks run concurrently and finish asynchronously, a task may draw only on outcomes of tasks that have already completed. Let $\mathcal{P}(t)$ denote the set of tasks completed before task t launches. The causal history of t comprises two feature vectors, each summarizing prior outcomes:

$$\mathbf{h}_{m,t} = \phi_m\big(\{\, y_\tau : \tau \in \mathcal{P}(t) \,\}\big),$$
$$\mathbf{h}_{c,t} = \phi_c\big(\{\, c_\tau : \tau \in \mathcal{P}(t),\ q_\tau = 1 \,\}\big),$$

where ϕ_m and ϕ_c are summarization maps. Peak-memory history draws on every completed task; consumption history draws only on those that were audited. We write $\mathbf{h}_t = [\mathbf{h}_{m,t}, \mathbf{h}_{c,t}]$. Which prior tasks each summary includes is a modeling choice addressed in Section III; the exact fields of ϕ_m and ϕ_c appear in the technical report [9].

At launch, a binary decision $q_t \in \{0, 1\}$ determines whether task t is instrumented. If $q_t = 1$ the task is audited, and both y_t and c_t enter the histories of later tasks; if $q_t = 0$, only y_t does, and c_t is never observed. Auditing incurs overhead, so the expected fraction of audited tasks is capped by a budget $B \in (0, 1]$:

$$\frac{1}{N} \sum_{t=1}^{N} \mathbb{E}[q_t] \le B. \tag{1}$$

We seek a prelaunch allocation function f and an auditing policy π,

$$\hat{y}_t = f(\mathbf{s}_t, \mathbf{h}_t), \qquad q_t = \pi(\mathbf{s}_t, \mathbf{h}_t, \hat{y}_t).$$

The allocation objective is asymmetric: underallocation ($\hat{y}_t < y_t$) risks an OOM abort and is penalized more heavily than overallocation, which only reserves capacity that goes unused. The policy π must stay within the auditing budget.

Because c_t and y_t become available only after completion, auditing task t cannot improve its own request; its measurements inform later tasks alone. The audit decision therefore uses only prelaunch information, including the request $\hat{y}_t$ just issued.

III. CAMP Design

We first describe how CAMP computes $\hat{y}_t$, then (Section III-E) the gates that determine the auditing policy π. Table II summarizes the additional notation.

TABLE II
NOTATION USED IN THE PROPOSED SOLUTION.

Symbols	Meaning
$K,\ I_i,\ n_i$	Number of scopes, scope i, and its minimum support threshold
$\mathcal{C}_{I_i}(t),\ \ell(t),\ p(i)$	Compatible completed-task pool, routed scope index, and parent scope
$b_t,\ r_\tau$	Base memory estimate and completed-task log residual
$R_i(t),\ \alpha,\ \widehat{Q}_\alpha$	Scope residual set, residual quantile level, and empirical quantile operator
$\eta_i,\ \rho_t$	Local confidence weight and blended residual correction
$p_t^r,\ \hat{\delta}_t$	Underallocation probability and conditional deficit
S_t^{risk}	Audit risk score

CAMP is designed around two core empirical observations. First, although a task's true data consumption c_t cannot be observed by the scheduler before launch, it can be estimated using available prelaunch features and memory metrics. This lightweight, learned proxy $\hat{c}_t$ serves as a supporting signal for peak memory estimation.

Second, accurate memory allocation requires learning at the correct level of task specificity. Frequently repeated pipeline steps often possess dozens of nearly identical historical runs to learn from, whereas a newly deployed workflow stage may have none. CAMP therefore organizes completed outcomes at

multiple levels of specificity and preserves their summaries as model features.

Central to the design of CAMP is a hierarchical system of *scopes* and model selection per workflow-process pair. *Scopes* determine what evidence a task receives: they organize completed tasks into pools of decreasing similarity, and their summaries enter every model as features. *Workflow–process pairs* determine which upper-tail model serves a task: tail behavior is process-specific, so each pair with enough training data gets its own tail ensemble. In other words, scopes govern evidence; and workflow-process pairs govern model selection.

Figure 1 shows how these mechanisms combine. For a pending task, CAMP (i) estimates the bytes the task will read, (ii) predicts a set of peak-memory quantiles, (iii) selects one quantile as an uncalibrated base estimate, and (iv) inflates that base by a residual correction to produce the request. Stages (i) and (ii) are learned models, stage (iii) is fixed offline by policy selection, and stage (iv) draws on residual history accumulated online. Scope summaries enter as features at stages (i) and (ii), and the routed scope supplies the residual pool at stage (iv); the workflow–process pair selects the upper-tail ensemble used at stage (ii).

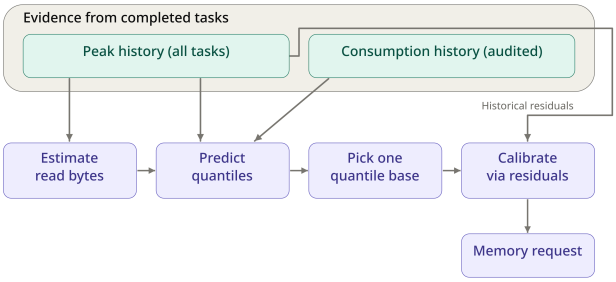


Fig. 1. CAMP allocation pipeline: consumption estimation, peak-memory quantile prediction, base selection, and residual calibration.

A. Evidence: scopes and causal history

Let $\mathcal{P}(t)$ denote the tasks that completed before task t launches. CAMP defines K scopes $I_0, \ldots, I_{K-1}$, ordered from most to least specific. For each scope i, the pool $\mathcal{C}_{I_i}(t) \subseteq \mathcal{P}(t)$ contains the completed tasks satisfying that scope's compatibility criterion, which ranges from an exact prior execution of the same task instance to a match on hardware profile alone. Let n_i be scope i's minimum support threshold. A task is routed to the strictest scope whose pool clears its threshold:

$$\ell(t) = \min\left\{\, i \in \{0, \ldots, K-1\} : |\mathcal{C}_{I_i}(t)| \geq n_i \,\right\}. \quad (2)$$

If no scope clears its threshold, the task receives a bootstrap default reservation.

The routed scope $\ell(t)$, recorded as `history_scope`, identifies the most trustworthy evidence available for t, and it selects the residual pool used in calibration. It does *not* restrict which pools contribute features. CAMP computes a summary block for every scope—medians, upper-tail percentiles, spreads, and sample support over the pool's peak-RSS values, and, where tasks were audited, over their consumption values—and concatenates all K blocks into $\mathbf{h}_{m,t}$ and $\mathbf{h}_{c,t}$, instantiating ϕ_m and ϕ_c of Section II.

A task routed to a strict scope therefore still sees the broader, more stable statistics of looser scopes, and can learn how much weight each deserves. Because at most a fraction B of tasks are audited, consumption blocks are sparse by construction; empty blocks are imputed and flagged rather than zero-filled, since zero is itself a plausible memory value. Field definitions, confidence metrics, and imputation mechanics appear in the technical report [9].

B. Stage 1: estimating consumed bytes

The consumption estimator g predicts how many bytes the pending task will read:

$$\hat{c}_t = g(\mathbf{s}_t, \mathbf{h}_{m,t}). \quad (3)$$

Because c_τ is observed only when $q_\tau = 1$, g trains on audited tasks alone, using $\{(\mathbf{s}_\tau, \mathbf{h}_{m,\tau}) \mapsto c_\tau : q_\tau = 1\}$ as its training set. Auditing does not improve the audited task's own request, but it supplies the only labels from which $\hat{c}_t$ can be learned for every later task.

g is global rather than scope- or pair-specific, for two reasons. Audited tasks are a minority of all tasks, so partitioning them further would leave each model data-starved; and consumption is a far more regular function of static features than peak memory is, so pooling costs little (Section V). g is a three-member LightGBM ensemble trained at $q = 0.50$ in $\log(1 + c)$ space, with $\hat{c}_t$ taken as the median of the three members. Consumption history $\mathbf{h}_{c,t}$ is deliberately excluded from g's inputs: it measurably degrades $\hat{c}_t$ (Section V).

C. Stage 2: predicting peak-memory quantiles

Peak memory is heavy-tailed, so CAMP predicts a set of quantiles rather than a point estimate, training under pinball loss at levels $\mathcal{Q} = \{0.50, 0.90, 0.95, 0.99, 0.995\}$ with y_t as target:

$$\left(\mathbf{s}_t,\, \mathbf{h}_t,\, \hat{c}_t\right) \longmapsto \{Q_q\}_{q \in \mathcal{Q}}. \quad (4)$$

The pinball objective is what encodes the asymmetry of Section II: an upper-quantile model is penalized far more for falling below the observed peak than above it.

The central estimate $Q_{0.50}$ comes from a global ensemble, while $Q_{0.90}$ through $Q_{0.995}$ come from ensembles trained per workflow–process pair, with a globally trained upper-tail ensemble as fallback for workflow–process pairs unseen during development.The grouped cross-fitting procedure used to generate leakage-free development predictions, including the split-group fallback criterion, is described in Section IV-C. The asymmetry is deliberate: central behavior is largely explained by input size and consumption, which transfer across processes, so the median benefits from pooling. On the contrary, tail behavior is driven by process-specific effects—buffer strategies, threading, allocator behavior under particular inputs—which do not transfer, so the tail benefits from specialization. As in Stage 1, each quantile is the median of three

seeded members, and a sorting pass after inference removes quantile crossing.

D. Stages 3 and 4: base selection and calibration

CAMP therefore does not assume any particular quantile is safe. Instead it treats the choice of quantile and the size of the correction applied to it as a single decision, resolved empirically on development data. For each completed task τ and each candidate quantile level q, CAMP records the log residual between the observed peak and the prediction at that level,

$$r_\tau = \log\big(y_\tau / b_\tau\big), \tag{5}$$

where b_τ is the candidate's prediction for τ. A positive residual means the candidate fell short. The log form makes the correction multiplicative and scale-free, so a systematic 10% shortfall is treated identically at 500 MiB and at 50 GiB rather than being dominated by the largest tasks.

Each candidate policy pairs a quantile level q with a residual quantile level α, giving a corrected request. CAMP evaluates every candidate on held-out development rows, retains those achieving at least 99% coverage overall and 97.5% within every workflow, and among the survivors selects the one with the smallest request-to-peak ratio. This matters because the cheapest safe policy is not generally the highest quantile: a moderate quantile with a well-estimated correction frequently covers as reliably as $Q_{0.995}$ while reserving substantially less memory. The selected pair is frozen before evaluation, and b_t denotes the base estimate it produces at inference.

Calibration then applies the correction online. Residuals are aggregated per scope; for scope I_i, let $R_i(t) = \{r_\tau : \tau \in \mathcal{C}_{I_i}(t)\}$ be the residuals available before t, and let $p(i)$ be the parent scope in the hierarchy. With $i = \ell(t)$, the correction blends the routed scope with its parent,

$$\rho_t = \eta_i \, \widehat{Q}_\alpha\big(R_i(t)\big) + (1 - \eta_i) \, \widehat{Q}_\alpha\big(R_{p(i)}(t)\big),$$

where $\widehat{Q}_\alpha$ is the empirical α-quantile and η_i a confidence weight increasing in residual support and decreasing in residual spread. Blending toward the parent is what prevents a scope with three residuals from calibrating on noise. The final request is

$$\hat{y}_t = \Big\lceil \, b_t \exp\big(\max(0, \rho_t)\big) \, \Big\rceil_{\mathrm{MiB}}. \tag{6}$$

The $\max(0, \rho_t)$ term makes calibration one-sided: a history of over-provisioning never shrinks a request, reflecting the asymmetry of the allocation objective. Rounding to whole MiB is an allocation-granularity requirement, not a safety margin; CAMP applies no fixed floor.

E. Selective Auditing

Full eBPF auditing supplies the consumed-data signal that extends $\mathbf{h}_{c,t}$, but auditing every task incurs the overhead measured in Section V. The auditing policy π therefore decides, before launch, which pending tasks should spend the budget B. An audit cannot help the task that produces it i.e., its value begins only when the record becomes history for later tasks.

Each pending task is scored from an audit-feature vector $\mathbf{x}_t$ drawn from the same prelaunch-legal information available to the allocator: $\mathbf{s}_t$, $\mathbf{h}_t$, the active request, and the model's quantiles. The task's own y_t, c_t, runtime, and outcome are excluded; they appear only as labels when scoring models are trained on completed tasks. Because score scales differ across workflows, learned probabilities are converted to a within-workflow percentile rank $\mathrm{rank}_w(\cdot) \in (0, 1]$ over pending tasks of the same workflow.

The budget is dominated by a single question: *is the active memory request likely to be too small?* Tasks whose requests are about to fail re- main valuable audit targets because, after completion, their measurements concentrate future consumed-data history where the active allocator was weakest. CAMP estimates this with two lightweight models trained on completed tasks: a classifier for the underallocation probability $p_t^r = \Pr(y_t > \hat{y}_t \mid \mathbf{x}_t)$, and, fitted only on completed underallocated tasks, a regressor for the conditional deficit $\hat{\delta}_t = \mathbb{E}[y_t - \hat{y}_t \mid y_t > \hat{y}_t, \mathbf{x}_t]$. The *risk score*

$$S_t^{\mathrm{risk}} = 0.70 \; \mathrm{rank}_w(p_t^r) + 0.30 \; \mathrm{rank}_w(p_t^r \, \hat{\delta}_t) \tag{7}$$

ranks a task highly when failure is likely, when its expected deficit is large, or both. At each decision point, CAMP allocates $\lfloor B \, |U| \rfloor$ audits over the pending set U, divides them among workflows in proportion to pending counts, and spends 80% of each workflow's quota on its highest-S_t^{risk} tasks.

The remaining 20% funds two secondary objectives: a *heavy-information* score selecting tasks whose consumed-data measurement is likely to improve future predictions, and a *discovery* allotment split between high-uncertainty tasks and random draws from under-sampled workflow/process groups. The random component protects against unmonitored drift and keeps inclusion probabilities non-degenerate; score definitions and quota mechanics appear in the technical report [9].

IV. Experimental Method

All results are available at [10].

A. Workflows, Datasets, and Platform

The matched offline cohort comprises 33,516 tasks across six workflows (Table III): "All"/"Test" denote total/held-out task counts; "Proc." counts distinct processes in the full cohort. Nextflow/nf-core [11], [12] runs on Slurm-based AWS ParallelCluster with 13 worker nodes (32 vCPUs, 256 GiB RAM each).

The custom workloads are deterministic: GATK runs repeated HaplotypeCaller scatters on paired reads; Minimap2 maps, indexes, and summarizes reads from fixed genomic windows. Sarek uses 30x coverage and GRCh38 interval scattering. Taxprofiler uses 5,998 Illumina metagenomic runs from an ENA snapshot; Seqinspector and Viralmetagenome use 3,000 from the same cohort with Kraken2 Standard-8. Spanning variant calling, alignment, taxonomic profiling, and metagenomic classification, these workloads vary the relationship among staged size, measured consumption, and peak memory.

TABLE III
MATCHED WORKFLOW COHORT.

Workflow	Version	Dataset	All	Test	Proc.
GATK	4.5.0.0 custom	Synthetic hg38 reads	500	75	1
Minimap2	custom	ONT-style reads	6,001	900	5
Sarek [13]	3.6.0	GIAB NA12878 WGS	6,012	902	25
Seqinsp. [14]	1.1.0	ENA metagenomes	6,001	900	3
Taxprof. [15]	2.0.1	ENA paired-end WGS	5,999	900	2
Viralmeta. [16]	1.1.3	ENA metagenomes	9,003	1,349	6
Total	–	–	33,516	5,026	–

TABLE IV
ACCESS AND CAMP MODEL INSTANTIATION.

Input parameters	Model outputs
Access: $\mathbf{x}_t^{\mathrm{Access}}$	$\{Q_q^{\mathrm{Access}}(t)\}_{q\in\mathcal{Q}}$ and calibrated request $\hat{y}_t^{\mathrm{Access}}$
CAMP: $\mathbf{x}_t^{\mathrm{CAMP}}$	$\{Q_q^{\mathrm{CAMP}}(t)\}_{q\in\mathcal{Q}}$ and calibrated request $\hat{y}_t^{\mathrm{CAMP}}$

B. CAMP Model Instantiation

The memory-allocation experiment evaluates two conditions on the same tasks. *Access* uses the static features and causal peak-memory history. CAMP adds consumption history from earlier audits and estimated current-task consumption:

$$\mathbf{x}_t^{\mathrm{Access}} = [\mathbf{s}_t, \mathbf{h}_{m,t}], \tag{8}$$

$$\hat{c}_t = g(\mathbf{s}_t, \mathbf{h}_{m,t}),$$

$$\mathbf{x}_t^{\mathrm{CAMP}} = [\mathbf{s}_t, \mathbf{h}_{m,t}, \mathbf{h}_{c,t}, \hat{c}_t]. \tag{9}$$

Table IV maps these inputs to the outputs produced by each condition using the notation from Section III.

For each condition, the routed model family produces peak-memory quantiles; calibration converts the selected quantile into a submitted request (Section III-D). Neither condition observes the current task's measured c_t or y_t before launch.

C. Training and Calibration Protocol

The 33,516-task cohort splits chronologically into 28,490 earlier development and 5,026 latest held-out tasks, preserving related task-instance, staged-input, and sample groups. Access and CAMP share tasks, peak-RSS target, model family, and preprocessing, with independent fitting and calibration using their legal inputs above. Peak RSS uses $\log(1 + y_t)$ (y_t in MiB); consumption targets use bytes.

Five-fold grouped out-of-fold models exclude each predicted task's split group from training. Split groups keep related tasks sharing a sample, input identity, or task-instance identifier in the same fold. Pair-specific out-of-fold predictions require at least two distinct split groups; pairs with fewer than two groups use global out-of-fold predictions. A deterministic identifier hash partitions predictions into 17,210 residual-reference rows for condition-specific calibration distributions and 11,280 policy-selection rows for evaluating candidates under Section III-D's criterion. Models then train on all development tasks. Sequential RQ2 evaluation freezes weights and policies; outcomes become available after requests, updating history and residuals for later tasks. Replay approximates recorded completion order; live CAMP uses only outcomes completed before task t's allocation decision time d_t.

Together, these controls support four research questions.

RQ1: eBPF and mmap signal. Compared with STRACE, how much explicit-read and memory-mapped page-fault activity does eBPF capture, and what runtime overhead does online full auditing introduce?

RQ2: Consumption-aware allocation. Does CAMP reduce underallocations and runtime-weighted unused memory relative to Access, and what reservation-cost tradeoffs result?

RQ3: Comparison with Sizey. On identical offline task assignments, how does CAMP compare with Sizey in underallocation, point error, and reserved-memory cost?

RQ4: Selective auditing. At equal eBPF audit counts, do CAMP's gates identify more underallocated, memory-heavy, and consumption-informative tasks than random selection?

D. Evaluation Protocol and Metrics

For first request $\hat{y}_t$, recorded peak RSS y_t, q50 prediction $\tilde{y}_t$, and runtime τ_t, the common allocation metrics are

$$u_t = \mathbf{1}[\hat{y}_t < y_t], \qquad \mathrm{Cov} = 1 - \frac{1}{N}\sum_t u_t,$$

$$\kappa = \frac{\sum_t \hat{y}_t}{\sum_t y_t}, \qquad W = \sum_t \tau_t(\hat{y}_t - y_t)_+,$$

$$\mathrm{MdAPE} = 100\,\underset{t}{\mathrm{median}}\frac{|\tilde{y}_t - y_t|}{y_t}.$$

Here u_t marks an underallocation, κ is the aggregate request-to-peak ratio, and W is unused memory-time, reported as relative to the baseline. We also report total requested memory as a percentage change.

RQ1. Matched no-audit, full-STRACE, and full-eBPF executions report explicit-read shares, eBPF mmap-fault bytes and their signal share, and runtime overhead $100(T_w^a/T_w^0 - 1)$ relative to no audit.

RQ2. Access and CAMP are compared using underallocation count, W. Robustness uses chronological grouped 30/70, 50/50, 70/30, 85/15, and 90/10 splits. Seeds 1996–2000 refit all models and policies at each split; coefficient of variation and t-based 95% intervals measure seed sensitivity on identical holdout rows.

Equal-budget control. On the primary holdout, we uniformly scale every Access first request by $k = \sum_t \hat{y}_t^{\mathrm{CAMP}} / \sum_t \hat{y}_t^{\mathrm{Access}}$, matching CAMP's aggregate request.

We retain the original predictions, histories, recorded peaks, and runtime convention, and compare underallocation counts and recorded unused memory-time.

mmap ablation. We compare explicit-read consumption with read+mmap consumption using identical tasks and model settings. Each variant constructs its history and trains its consumption and memory models, with independent development-only policy selection. Underallocations use each variant's own requests without budget matching.

RQ3. Sizey and CAMP receive identical tasks under Sizey's 9,141/21,337-task per-process split. Sizey's process-size eligibility rule removes approximately 3,000 records. Underallocation, coverage, κ, and W use each method's first request; q50 MdAPE uses each method's central estimate.

Matched operating points. We apply the same equal-budget rule to Sizey, retaining all predictions, histories, and measured peaks. Separately, we find each method's smallest uniform request multiplier attaining at least 95% observed coverage and compare total requests across all tasks.

RQ4. Similarly 9,141-task development set selects and freezes the three-gate policy before replaying 21,337 later tasks. Active-risk and heavy-information, and recall are compared with uniform and process-stratified random selection at equal counts. Budgets of 5–50% use 1,000 repetitions, with empirical 95% intervals for the random baslines.

Downstream allocation. We compare two identically initialized CAMP predictors with selective or random auditing at equal per-workflow audit counts. We test 5% - 50% budgets with five sampling seeds, predefining 20% as primary. We compare original total requests and predicted OOM events.

V. Results

A. RQ1: Audit Signal and Cost

Figure 2(a) places workflows on the y-axis and the percentage composition of the eBPF consumed-data signal on the x-axis. Mmap faults supply 50.7% of Minimap2's consumed-data signal and 16.9–19.9% for GATK and Viralmetagenome. Figure 2(b) shows that eBPF full-audit overhead is not uniformly lower than STRACE: it is similar or lower for GATK, Sarek, and Taxprofiler, but higher for the other workflows. Thus, eBPF's main benefit is visibility into mmap-backed consumption, while its variable full-audit cost motivates selective auditing.

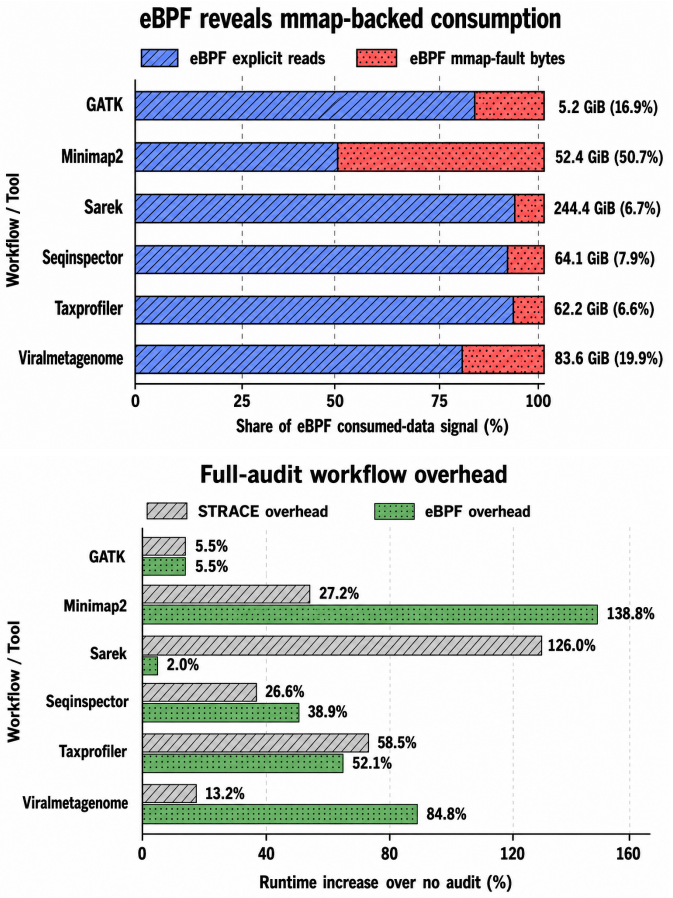


Fig. 2. RQ1 full auditing. (a) eBPF explicit-read and mmap-fault composition; labels give mmap bytes and signal share. (b) STRACE and eBPF workflow-runtime overhead over no audit.

B. RQ2: Consumed-Data Feedback

Figure 3 plots underallocations against development/holdout split: raw test counts (a) and CAMP rates per 1,000 tasks (b). Colored points represent five training seeds; diamonds and error bars show means and 95% confidence intervals; CV denotes coefficient of variation. CAMP underallocates fewer tasks than Access at all five splits in (a).

In (b), 50/50 has the lowest mean (7.4; CV 20.2%); 85/15 averages 12.1 with the highest relative variation (CV 67.4%); 70/30 averages 32.7 with the lowest (CV 5.9%). Means for 30/70 and 90/10 are 32.6 and 23.7. Seed stability thus need not imply low underallocation.

In the primary 85/15 run (seed 1996), CAMP reduces underallocations by 64.37% and unused memory-time by 37.18%, requesting 31.50% more memory.

Adding $\hat{c}_t$ to static features, peak-memory history, and consumption history reduces underallocations from 46 to 30 and total requests from 80,853 to 34,271 GiB (57.61%), but raises median peak-memory prediction error from 21.32% to 26.51%.

mmap ablation. Adding mmap-derived consumption reduces native underallocations from 32 to 30 in the primary comparison, indicating a modest allocation benefit at the selected native request levels.

Equal-budget comparison. Scaling Access by 1.315 matches CAMP's 34,277.55 GiB. budget and yields 2 underallocations versus CAMP's 31. CAMP uses 39.87 versus 1433.19 GiB-hours of recorded unused memory-time (72.2% less), showing a coverage-memory-time tradeoff.

C. RQ3: External Comparison with Sizey

Figure 4 compares Sizey and CAMP by underallocated task count (a) and median absolute percentage error (b); panel (c) reports request-to-peak ratio and unused memory-time relative to Sizey (100%). CAMP reduces underallocations from

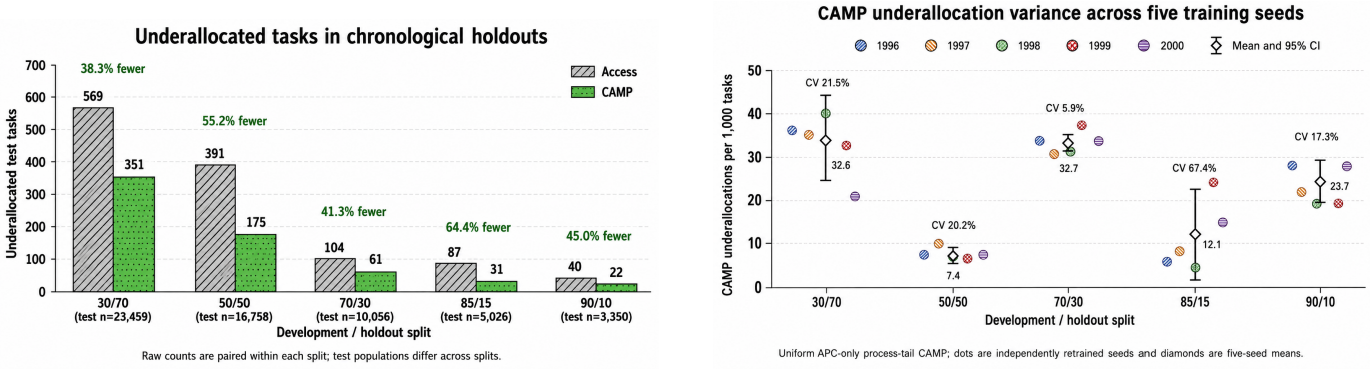


Fig. 3. RQ2 allocation safety and robustness. (a) Raw underallocation counts; holdout sizes differ by split. (b) CAMP underallocations per 1,000 tasks across five training seeds, with means, 95% confidence intervals, and coefficients of variation.

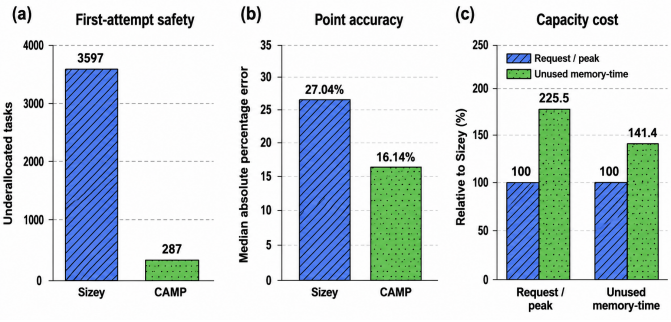


Fig. 4. 21,337-task comparison with Sizey: (a) underallocations; (b) median absolute percentage error; (c) request/peak and unused memory-time normalized to Sizey.

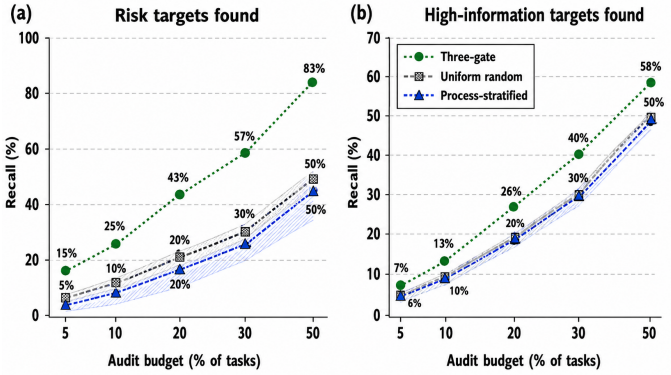


Fig. 5. RQ4 equal-count selective-audit recall. Curves are means over 1,000 repetitions; shading gives empirical 95% random-baseline intervals

3,597 to 287 (92.02%) and raises first-attempt coverage from 83.14% to 98.65%. Median absolute percentage error falls from 27.04% to 16.14% (40.31% improvement). However, its request-to-peak ratio and unused memory-time reach 225.5% and 141.4% of Sizey's, respectively: CAMP improves safety and point accuracy but reserves more capacity.

Matched operating-point comparison. Scaling Sizey's requests by 2.2546 matches CAMP's aggregate budget of 79,049.42 GiB, at which CAMP underallocates 287 tasks versus Sizey's 1,119 (74.4% fewer). At matched observed coverage of 95.004% (1,066 underallocations each), CAMP requests 77,818.52 GiB versus Sizey's 93,357.81 GiB (16.6% less).

D. RQ4: Selective Auditing

Figure 5 plots target recall against audit budget (percentage of tasks audited), with equal task counts across policies. Risk targets (a) have active requests below peak RSS; high-information targets (b) meet the heavy-information gate's joint memory-tail and consumption-utility label. At 5%, 10%, 20%, 30%, and 50% budgets, CAMP recalls 15%, 25%, 43%, 57%, and 83% of risk targets, respectively, versus 5%, 10%, 20%, 30%, and 50% for uniform random selection. High-information recall reaches 7%, 13%, 26%, 40%, and 58%, exceeding both equal-count random baselines at every displayed budget. The gates thus prioritize underallocation risk while identifying more tasks useful for refreshing consumption-aware models.

Downstream allocation. At the predefined 20% audit budget, results show 13.1% fewer mean predicted OOM events with selective auditing than random sampling (265.8 versus 305.8 across five seeds), with 2.4% more total requested memory.

VI. Related Work

Workflow memory prediction. Predictors use prelaunch inputs and completed tasks [17]. Ferreira da Silva et al. [18], [19] update runtime, disk, and peak-memory estimates online; Tovar et al. use historical allocation or reshape splittable analyses [8], [20]; Witt et al. optimize allocation cost through feedback [4], [21]. Sizey trains and reselects models by allocation quality [6]; Ponder selects sizing strategies [5]. Others predict time-varying memory [22], learn allocation actions [23], profile runtime cold starts [24], or use scheduler logs [7], [25]. CAMP augments completed-task history with selectively measured consumption, distinguishing consumed from staged data.

Tracing and provenance. System-call auditing costs vary by workload [26]. LPM [27] and CamFlow [28] capture whole-system provenance; CAMP uses verified eBPF

hooks [29] to attribute positive reads and file-backed mmap faults to tasks.

Budgeted measurement. Active learning selects labels within measurement budgets [30]. CAMP records all tasks' peak RSS but selectively acquires consumption labels. Its random lane uses recorded inclusion probabilities and capped Horvitz–Thompson weights [31] to reduce sampling bias. Deterministic risk/information selections fall outside this probability sample; evaluation therefore measures selection yield without claiming an unbiased complete audit set.

VII. Conclusion

CAMP shows that evidence granularity complements model selection: runtime consumption helps explain heavy-tail memory behavior beyond static metadata. To limit tracing overhead, it combines structured history with budgeted auditing, achieving higher risk and high-information recall than random sampling at equal audit counts [10].

Acknowledgment

This work is supported by NASA under grant NASA-AIST-21-0095

References

[1] A. B. Yoo, M. A. Jette, and M. Grondona, "SLURM: Simple Linux utility for resource management," in *Job Scheduling Strategies for Parallel Processing (JSSPP 2003)*, ser. Lecture Notes in Computer Science, D. Feitelson, L. Rudolph, and U. Schwiegelshohn, Eds., vol. 2862. Springer, 2003, pp. 44–60.

[2] D. Thain, T. Tannenbaum, and M. Livny, "Distributed computing in practice: The Condor experience," *Concurrency and Computation: Practice and Experience*, vol. 17, no. 2–4, pp. 323–356, 2005.

[3] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016.

[4] C. Witt, D. Wagner, and U. Leser, "Feedback-based resource allocation for batch scheduling of scientific workflows," in *2019 International Conference on High Performance Computing and Simulation (HPCS)*. Dublin, Ireland: IEEE, 2019, pp. 761–768.

[5] F. Lehmann, J. Bader, N. De Mecquenem, X. Wang, V. Bountris, F. Friederici, U. Leser, and L. Thamsen, "Ponder: Online prediction of task memory requirements for scientific workflows," in *2024 IEEE 20th International Conference on e-Science (e-Science)*. Osaka, Japan: IEEE, 2024, pp. 1–10.

[6] J. Bader, F. Skalski, F. Lehmann, D. Scheinert, J. Will, L. Thamsen, and O. Kao, "Sizey: Memory-efficient execution of scientific workflow tasks," in *2024 IEEE International Conference on Cluster Computing (CLUSTER)*. Kobe, Japan: IEEE, 2024, pp. 370–381.

[7] E. R. Rodrigues, R. L. F. Cunha, M. A. S. Netto, and M. Spriggs, "Helping HPC users specify job memory requirements via machine learning," in *Proceedings of the Third International Workshop on HPC User Support Tools (HUST '16)*. Salt Lake City, UT, USA: IEEE Press, 2016, pp. 6–13.

[8] B. Tovar and et. al., "A job sizing strategy for high-throughput scientific workflows," *IEEE Transactions on Parallel and Distributed Systems*, vol. 29, no. 2, pp. 240–253, 2018.

[9] N. Kandimalla, B. P. Vangala, and T. Malik, "CAMP: Consumption-aware memory prediction for scientific workflows—extended technical report," University of Missouri–Columbia, Tech. Rep., 2026. [Online]. Available: \url{https://radiant-systems-lab.github.io/pdfs/pdfs/pubs/IEEE_SCWorks_26.pdf}

[10] N. Kandimalla and T. Malik, "CAMP experimental artifacts and reproduction package," https://anonymous.4open.science/r/IEEE_WORKS_26_ARTIFACT-B120/README.md, 2026, accessed: 2026-08-08.

[11] P. Di Tommaso, M. Chatzou, E. W. Floden, P. P. Barja, E. Palumbo, and C. Notredame, "Nextflow enables reproducible computational workflows," *Nature Biotechnology*, vol. 35, no. 4, pp. 316–319, 2017.

[12] P. A. Ewels, A. Peltzer, S. Fillinger, H. Patel, J. Alneberg, A. Wilm, M. U. Garcia, P. Di Tommaso, and S. Nahnsen, "The nf-core framework for community-curated bioinformatics pipelines," *Nature Biotechnology*, vol. 38, no. 3, pp. 276–278, 2020.

[13] nf-core community, "nf-core/sarek release 3.6.0," GitHub release: https://github.com/nf-core/sarek/releases/tag/3.6.0, Oct. 2025, accessed: 2026-08-08.

[14] ——, "nf-core/seqinspector release 1.1.0," GitHub release: https://github.com/nf-core/seqinspector/releases/tag/1.1.0, Jul. 2026, accessed: 2026-08-08.

[15] ——, "nf-core/taxprofiler release 2.0.1," GitHub release: https://github.com/nf-core/taxprofiler/releases/tag/2.0.1, Jun. 2026, accessed: 2026-08-08.

[16] ——, "nf-core/viralmetagenome release 1.1.3," GitHub release: https://github.com/nf-core/viralmetagenome/releases/tag/1.1.3, Jul. 2026, accessed: 2026-08-08.

[17] J. Bader, K. West, S. Becker, S. Kulagina, F. Lehmann, L. Thamsen, H. Meyerhenke, and O. Kao, "Predicting the performance of scientific workflow tasks for cluster resource management: An overview of the state of the art," *arXiv preprint arXiv:2504.20867*, 2025.

[18] R. F. Da Silva, G. Juve, E. Deelman, T. Glatard, F. Desprez, D. Thain, B. Tovar, and M. Livny, "Toward fine-grained online task characteristics estimation in scientific workflows," in *Proceedings of the 8th Workshop on Workflows in Support of Large-Scale Science (WORKS '13)*. New York, NY, USA: ACM, 2013, pp. 58–67.

[19] R. F. Da Silva, G. Juve, M. Rynge, E. Deelman, and M. Livny, "Online task resource consumption prediction for scientific workflows," *Parallel Processing Letters*, vol. 25, no. 3, 2015, art. no. 1541003.

[20] B. Tovar, B. Lyons, K. Mohrman, B. Sly-Delgado, K. Lannon, and D. Thain, "Dynamic task shaping for high throughput data analysis applications in high energy physics," in *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2022, pp. 346–356.

[21] C. Witt, J. van Santen, and U. Leser, "Learning low-wastage memory allocations for scientific workflows at IceCube," in *2019 International Conference on High Performance Computing and Simulation (HPCS)*. IEEE, 2019, pp. 233–240.

[22] J. Bader, N. Diedrich, L. Thamsen, and O. Kao, "Predicting dynamic memory requirements for scientific workflow tasks," in *2023 IEEE International Conference on Big Data (BigData)*. IEEE, 2023, pp. 182–189.

[23] J. Bader, N. Zunker, S. Becker, and O. Kao, "Leveraging reinforcement learning for task resource allocation in scientific workflows," in *2022 IEEE International Conference on Big Data (BigData)*. IEEE, 2022, pp. 3714–3719.

[24] J. Bader, F. Lehmann, L. Thamsen, U. Leser, and O. Kao, "Lotaru: Locally predicting workflow task runtimes for resource management on heterogeneous infrastructures," *Future Generation Computer Systems*, vol. 150, pp. 171–185, 2024.

[25] M. Tanash, B. Dunn, D. Andresen, W. Hsu, H. Yang, and A. Okanlawon, "Improving HPC system performance by predicting job resources via supervised machine learning," in *Proceedings of the Practice and Experience in Advanced Research Computing on Rise of the Machines (Learning) (PEARC '19)*. New York, NY, USA: ACM, 2019, pp. 1–8.

[26] L. Zeng, Y. Xiao, and H. Chen, "Linux auditing: Overhead and adaptation," in *2015 IEEE International Conference on Communications (ICC)*. London, UK: IEEE, 2015, pp. 7168–7173.

[27] A. Bates, D. J. Tian, K. R. B. Butler, and T. Moyer, "Trustworthy whole-system provenance for the Linux kernel," in *24th USENIX Security Symposium (USENIX Security 15)*. Washington, D.C., USA: USENIX Association, 2015, pp. 319–334.

[28] T. Pasquier, X. Han, M. Goldstein, T. Moyer, D. Eyers, M. Seltzer, and J. Bacon, "Practical whole-system provenance capture," in *Proceedings of the 2017 Symposium on Cloud Computing (SoCC '17)*. Santa Clara, CA, USA: ACM, 2017, pp. 405–418.

[29] B. Gbadamosi, L. Leonardi, T. Pulls, T. Høiland-Jørgensen, S. Ferlin-Reiter, S. Sorce, and A. Brunström, "The eBPF runtime in the Linux kernel," *arXiv preprint arXiv:2410.00026*, 2024.

[30] B. Settles, "Active learning literature survey," University of Wisconsin–Madison, Computer Sciences Technical Report 1648, 2009.

[31] D. G. Horvitz and D. J. Thompson, "A generalization of sampling without replacement from a finite universe," *Journal of the American Statistical Association*, vol. 47, no. 260, pp. 663–685, 1952.